

Planned technologies:

- Framework: Next.js 16 (App Router), TypeScript
 - **Implemented as written:** Every page and API route uses the Next.js 16 App Router with TypeScript throughout. All files are .tsx or .ts.
 - **Note:** Next.js 16 introduced breaking changes from tutorials: middleware.ts renamed to proxy.ts, exported function must be named proxy, and highly interactive pages require `ssr: false` via `next/dynamic` to avoid hydration errors.
- Styling: Tailwind CSS v4
 - **Implemented as written.** All styling is Tailwind v4. No external UI component libraries were used, everything is custom built.
- Auth & DB: Supabase (Auth + PostgreSQL)
 - **Implemented as written.** Supabase Auth handles email/password signup and login. PostgreSQL stores trips (trips table) and usage tracking (usage_records table). Row Level Security policies ensure users can only access their own data.
 - lib/supabase/client.ts — browser-side client
 - lib/supabase/server.ts — server-side client
 - Email confirmation disabled to avoid Supabase free tier rate limits (3 emails/hour)
- AI: Anthropic Claude API (claude-sonnet)
 - **Implemented as written.** Uses claude-sonnet-4-5 via @anthropic-ai/sdk. System prompt enforces JSON-only output. Implementation in app/api/generate/route.ts.
 - **Added beyond proposal:** hallucination check after generation verifies place_name fields in activities against enrichment context. Attaches dataConfidence object (hitRate, flag: high|medium|low) to itinerary. Results page shows confidence badge.
- External APIs:
 - Google Places API
 - **Working:** used for city/destination autocomplete in the form (app/api/places/route.ts) and for attraction data in enrichment (app/api/enrich/route.ts)
 - Distance Matrix for drive time from airport to destination
 - National Park Service (NPS) API
 - **Working:** fetches nearby national parks by state code in app/api/enrich/route.ts

- RapidAPI hotel
 - **Working:** fetches real hotel prices and names in app/api/enrich/route.ts
 - Two-step lookup: location search → hotel search with checkin/checkout dates
 - Returns real hotel names and per-night prices
 - Hotels panel displayed on results page (app/trips/[id]/page.tsx)
 - RapidAPI flight
 - **New** app/api/flights/autocomplete/route.ts — airport search, returns US-only commercial airports
 - **New** app/api/flights/search/route.ts — roundtrip search with dynamic nearest-airport fallback
 - Step 2.5 "Flight Details" added to form (conditional on Flying/Either/Flexible)
 - Flight results displayed on results page with airline, times, prices, segments
 - Drive time from nearest airport shown when destination has no direct airport
 - Limitation: API only has data ~60 days out; falls back to Google Flights link for future dates
 - Deployment: Vercel
 - **Implemented as written.** Auto-deploy from GitHub main branch. Live at easy-trip-steel.vercel.app. Environment variables configured in Vercel dashboard.
- First deliverable:** Original: A logged in user fills out the multi step plan form, submits it, and receives a structured AI generated day by day itinerary saved to their account that they can look at later.
- **Implemented as written.** The complete end-to-end flow works:
 - User signs up or logs in at /auth
 - Fills out the 4-step form at /plan (destination with Google Places autocomplete, dates, budget, group size, travel style, transportation, accommodation, must-haves)
 - Submits, triggers POST /api/plan which orchestrates enrichment → generation → save

- Receives a structured JSON itinerary generated by Claude, enriched with real hotel prices (Booking.com), attractions (Google Places), and national park data (NPS)
- Redirected to /trips/[id] where the full day-by-day itinerary is rendered
- Trip is saved to Supabase and accessible from the dashboard at any time

Rough Architecture for First Deliverable:

- AuthGate (Supabase Auth)
 - Input: email + password
 - Output: active session, JWT in cookies
 - Effect: creates a user row on sign-up; locks down /plan and /trips routes via Next.js middleware so randos can't hit them
 - **Implemented as written.**
 - Login/signup UI: app/auth/page.tsx
 - Route protection: proxy.ts (Next.js 16 uses proxy.ts instead of middleware.ts, functionally identical)
 - Session management: lib/supabase/client.ts (browser) and lib/supabase/server.ts (server)
 - Added beyond proposal: lockout after 5 failed attempts, forgot password flow, duplicate email detection, show/hide password toggle
- PlanForm (multi-step UI)
 - Input: user filling out 4 steps: destination, dates, group size, interests
 - Output: a TripRequest object { destination, startDate, endDate, groupSize, interests[] }
 - Effect: validates each step before letting you move forward; fires off the request on the last step
 - **Steps implemented:**
 - **Step 1: Destination & Dates (Google Places autocomplete, up to 5 stops, date validation)**
 - **Step 2: Customize Your Experience (travel style, transportation, accommodation)**

- Step 2.5: Flight Details (conditional — only shown when Flying or Either/Flexible selected; departure airport autocomplete, cabin class)
- Step 3: Budget & Group Size (total budget, per-person calculation, flight budget, accommodation budget with full trip cost validation)
- Step 4: Must Haves & Submit
- Added beyond proposal:
 - Google Places autocomplete for city/destination search
 - Up to 5 destinations with live route preview
 - Budget breakdown with per-person calculations, accommodation cost over full trip duration, remaining budget display
 - localStorage auto-save so refreshing restores progress
 - Restart button with confirmation dialog
 - Loading overlay with rotating messages on submit
 - Dynamic progress bar (4 or 5 steps based on transportation choice)
- EnrichmentFetcher (server-side, runs in parallel)
 - Input: TripRequest
 - Output: EnrichedContext { nearbyParks: Park[], placeSuggestions: Place[], hotelOptions: Hotel[] }
 - Effect: hits NPS for nearby parks, Google Places for attractions/restaurants, RapidAPI for hotels, all at once. Then bundles the results to feed into Claude as grounding context
 - **Implemented as written.**
 - File: app/api/enrich/route.ts
 - Uses Promise.allSettled() so failures in any one source don't block the others
 - NPS returns parks filtered by state code with names, descriptions, entrance fees
 - Google Places returns top-rated attractions at the destination
 - Booking.com returns real hotel options with per-night prices
 - Flights: calls /api/flights/search if departureCode provided (added for final submission)
- ItineraryGenerator (Claude API call)

- Input: TripRequest + EnrichedContext
- Output: a structured Itinerary { days: Day[], summary: string, tips: string[] } as JSON
- Effect: one Claude call with a system prompt telling it to return only valid JSON; parses and validates before doing anything with it
 - **Implemented as written.**
 - File: app/api/generate/route.ts
 - Uses claude-sonnet-4-5 via @anthropic-ai/sdk
 - System prompt enforces JSON-only output — no markdown, no backticks, no explanation text
 - Strips accidental markdown fences before JSON.parse()
 - Returns { itinerary, tokensUsed }
 - Input validation returns 400 for missing fields
 - Added beyond proposal: token count returned to
 - CostGuard for accurate usage recording
 - **Added beyond proposal:**
 - Hallucination check: verifies place_name fields in generated activities against enrichment context using fuzzy matching. Calculates hit rate and attaches dataConfidence: { hitRate, flag } to itinerary. Flag is high (≥70% verified), medium (40-70%), or low (<40%).
 - Flight context in prompt: when departureCode provided, Claude receives real flight options and is instructed to include specific airline/time/price in Day 1 travel activity
- CostGuard (wrapper around ItineraryGenerator)
 - Input: user ID + their token usage for the current month from Supabase
 - Output: go or no-go
 - Effect: checks if the user is under their monthly token ceiling before calling Claude; bumps the counter after a successful call; shows a friendly error if they've hit the limit
 - **Implemented as written.**
 - File: lib/costGuard.ts
 - checkAndReserve(userId, estimatedTokens), reads usage_records, returns { allowed, remaining }

- recordUsage(userId, tokensUsed), increments tokens_used after successful generation
- Ceiling: 50,000 tokens/month per user
- 429 response with friendly message if limit is hit
- Known limitation: not atomic, race condition possible under high concurrency. Documented in DESIGN.md. Acceptable at current usage scale.
- Fix applied (course staff + peer feedback):
 - Original implementation had a race condition — two simultaneous requests could both pass the ceiling check. Fixed by replacing two-step read/write with atomic PostgreSQL stored procedure check_and_increment_usage. The UPDATE ... WHERE tokens_used + estimated <= ceiling is atomic at the database level. Comment updated to reflect this is no longer an "MVP" limitation.
- TripStore (Supabase DB layer)
 - Input: user ID + the finished Itinerary
 - Output: a tripId
 - Effect: writes a row to the trips table so it can be pulled up later on the dashboard
 - **Implemented as written.**
 - File: lib/tripStore.ts
 - saveTrip() inserts into trips table and returns the new id
 - Added beyond proposal: deleted_at column added for soft delete / trash bin feature
- ResultsPage (/trips/[id])
 - Input: tripId from the URL
 - Output: the full rendered itinerary
 - Effect: fetches the trip from Supabase and renders it as day cards with activities, places, and tips. only visible to the user who made it
 - **Implemented as written.**
 - File: app/trips/[id]/page.tsx
 - Fetches trip client-side, verifies user_id matches logged-in user

- Renders: hero section, trip summary, day cards with color-coded activity timeline (activity/food/lodging/travel), travel tips, action buttons
- Added beyond proposal: share button that makes trip public and copies a shareable URL to clipboard

- Added beyond proposal:

- Share button (sets `is_public = true`, copies URL to clipboard)
- Data confidence badge (high/medium/low based on hallucination check)
- Flight options panel with real live data
- Hotels panel with Booking.com prices

After first deliverable goals:

- Trip dashboard: show all your saved trips in one place with destination, dates, and a card layout; let you delete old ones
 - **Implemented as written, with additions.**
 - File: `app/trips/page.tsx`
 - Responsive card grid (3/2/1 columns by breakpoint)
 - Added beyond proposal: soft delete with 7-day trash bin, restore from trash, auto-purge on page load, badge count on trash icon
- Regenerate a trip: tweak your preferences and re-run the AI without having to fill out the whole form again
 - **No longer planned for initial submission. Will be considered for final submission if time permits.**
- Hotel & flight panel: show RapidAPI hotel and flight results right next to the itinerary so you can actually book
 - **Fully implemented for final submission:**
 - Hotels panel: live Booking.com prices displayed on results page
 - Flights panel: real flight data from Flights Scraper Data API with airline, times, prices, segments
 - Both integrated into `itinerary.json` at save time so they persist with the trip
- Places enrichment: pull in photos, ratings, and hours from Google Places inline so the itinerary feels real, not just text

- **Partially implemented.** Attraction names and ratings are fetched from Google Places and used by Claude in generation. Photos and hours are not yet pulled. Could be planned for final submission.
- NPS highlights: surface nearby national parks with descriptions and links to the official NPS pages
 - **Implemented as written.** NPS data (park name, description, entrance fee, official URL) is fetched in `app/api/enrich/route.ts` and passed to Claude as context. Claude incorporates park details directly into the itinerary activities and tips.
- Shareable trips: generate a public read-only link that anyone can view without logging in
 - **Implemented.**
 - Share button on results page sets `is_public = true` in Supabase and copies the share URL to clipboard
 - Public view: `app/trips/[id]/share/page.tsx` — no auth required, read-only, shows a "Plan your own trip" banner
 - RLS policy updated to allow public read access to trips where `is_public = true`
- Usage UI: let users see how many tokens they've used this month and how much budget they have left
 - **Implemented.** A usage indicator component showing tokens used / remaining with a color-coded progress bar is added to the `/plan` page header.